# 技術スタック採用方針

# AI時代のデータ基盤選定

LLMは十分に賢い。鍵は適切なデータを、適切な権限で渡せる基盤であること。

## Foundry移行とAI活用の関係

今回のRFPの主目的はFoundry基盤の確実な移行。

しかし移行先の基盤が、将来のAI活用の選択肢を決める。

**「今選ぶ基盤」が「将来の AI活用の天井」になりうる。**

| | |
|---|---|
| **変わるもの** | AI関連の技術スタック。LLMの最適解は半年で変わる。Agent基盤も発展途上。**特定のLLM・ベンダーに絞り込みすぎない設計が重要** |
| **変わりにくいもの** | データが正しく溜まり、加工され、探せる状態。ガバナンスが効いていること。**技術が変わっても価値が残る土台** |

## 基盤が拓く AI活用の可能性

1. 介護記録テキストの自動要約・分析
2. カメラ映像のAI解析（転倒検知等）
3. ケアプラン支援（過去データからの最適提案）
4. 全国施設横断のナレッジ共有・検索
5.職員の離職要因分析

**SOMPOケアの多数施設・大量の介護データは、AI進化の恩恵を最も受けられる領域**

# AI活用基盤に求められる 7つの観点

AI観点では、①②は今回の選定基準。③～⑥は将来の拡張余地をFPで確認。⑦は将来検討として記載する

| 観点 | 分類 | 今回の検討項目 | 将来的なAI観点の論点 |
|---|---|---|---|
| ① 連携性 | 今回選定 | 多様なデータソースに対応可能か | グループ会社間連携（マルチクラウド）・リアルタイム連携 |
| ② 安全性 | 今回選定 | データアクセス制御・監査ログ | AIのID・権限、ガードレール |
| ③ 進化性 | 余地確認 | 将来の機能追加しやすいか | マルチLLM、評価パイプライン、継続的改善の設計 |
| ④ 実効性 | 余地確認 | 非構造化データの格納・加工に対応可能か | AIが活用しやすい構造化、Agent設計、応答速度帯 |
| ⑤ 運用性 | 余地確認 | ログ・トレース・コスト管理 | Agentの挙動追跡 |
| ⑥ 開発速度 | 余地確認 | AI開発環境の標準構成案が具体か | PoCから本番移行までの開発標準化・運用ルール整備 |
| ⑦ 規模 | 将来検討 | 対象外 | AI/Agent増加時の標準化、責任分界、コスト統制 |

※⑦規模に関しては、AI/Agentの数や運用形態が固まる前に規模前提の標準化を進めても過剰設計になりやすいため、現時点では将来検討でよい

※将来的な論点の詳細に関しては、後述のAI活用に向けた将来論点で整理

次に、基盤が管理すべき資産の全体像を整理し、これらの観点から設計方針とベンダー評価を行う。

# 管理すべきデータ・資産の全体像

基盤が管理すべき範囲はデータとパイプラインに留まらず、メタデータ、権限、品質、監査、提供資産まで含む。将来のAI活用では、さらに非構造化データやモデル関連資産まで対象が拡大する。

| 区分 | 今回のRFP範囲 | 将来追加される管理対象 |
|---|---|---|
| **データ** | 構造化データ（テーブル・ビュー） | 非構造化（テキスト・音声・画像）、ベクトル |
| **パイプライン** | 既存パイプライン群、SQL変換 | 非構造化の加工パイプライン（チャンク化・ベクトル化） |
| **品質・運用** | パイプライン実行ログ、監査ログ、品質ルール、チェック結果、 | AI推論ログ、Agent実行トレース、コストログ、AI入出力 |
| **モデル・AI** | —（枠確保のみ※） | MLモデル、プロンプト/RAG資産、評価データ・中間生成物・推論結果 |
| **メタデータ** | スキーマ、リネージ | 非構造化メタデータ、学習・評価データの管理 |
| **権限・統制** | ユーザー権限、属性ベースのアクセス制御、データ分類 | AIアクセス権（主体のID設計）、動的権限昇格 |

※今回はML/AI機能自体の実装は対象外だが、将来拡張を阻害しない前提条件は今回確認する

# 推奨アーキテクチャ・ソフトウェアの採用方針

7観点と資産全体像から判断原則を導き、設計方針を定める

## 判断原則 → 設計方針

**原則1（①連携性・④実効性）**
多様なデータの統合管理
構造化+非構造化を同一基盤で

**統合データ基盤**
メダリオン設計（Bronze→Silver→Gold）でデータ品質を段階的に向上。データレイク /DWH（Data Lakehouse）を単一基盤で実現

**原則2（②安全性・⑤運用性）**
データ・資産のガバナンス
全資産の可視化・追跡・権限統制

**統合カタログ + リネージ + 権限管理**
資産全体像を一元管理。MLモデル・Agentまで拡張可能な設計
人のAIもどういつ基盤で権限を管理

**原則3（③進化性）**
ロックイン回避
ベンダー交渉力・移行コスト最小化

**オープン標準**
Delta/Icebergでポータビリティ確保。マネージドサービスで運用負荷を抑制。マルチ LLM対応

## 機能要件の方針

| 層 | ベンダーに求める方針 |
|---|---|
| インテグレーション | 多様なソース対応（PDF/Teams/SF/IoT） |
| ストレージ | オープンフォーマット。メダリオン構成。非構造化対応可 |
| オーケストレーション | 運用管理の簡便性 |
| 変換・処理 | 既存パイプラインの移植性。非構造化データ加工への拡張 |
| ガバナンス | 権限管理。自動リネージ。監査ログ |
| ML/AI | AI開発環境を将来追加しやすい設計 |
| 可視化 | Tableau活用。Workshop/Slate代替確認 |

# 6社RFI回答のレビュー（AI観点）

原則1・2・3に照らし、AI観点で6社のRFI回答を評価する

| 方針 | 評価項目 | Deloitte | Fujitsu | PwC | Hitachi | SF | Sky |
|---|---|---|---|---|---|---|---|
| 原則1 | 非構造化<br>データ対応 | ○ Unity Catalog<br>+OpenSearch | △ DI Basic内<br>具体設計なし | △ Step3先送り | △ Document AI<br>+Cortex Search | △ Data 360<br>松プラン限定 | ○ AI Functions+RAG<br>+AI-OCR |
| 原則2 | カタログ<br>・リネージ | ○ Unity Catalog<br>自動リネージ+品質 | ○ Unity Catalog<br>DI Basic内蔵 | ○ DataZone<br>リネージ+MDM | △ Snowflake内蔵<br>専用カタログなし | △ Data 360<br>松プラン限定 | ○ Unity Catalog<br>松プランのみ |
| 原則2 | 権限管理 | ○ Unity Catalog<br>RLS+列マスキング | △ DI Basic内<br>具体設計なし | ○ IAM+Cognito<br>RLS対応 | ○ Snowflake RLS<br>+マスキング | ○ Trust Layer<br>RLS+CRLS | ○ IAM（梅竹）<br>Unity Catalog RLS（松） |
| 原則3 | AI開発ロックイン | △ Delta Lake/MLflow<br>aws構成での<br>Dify依存に注意 | × AI層がKozuchi<br>独自依存度大 | ○ AWS標準サービス<br>移行先あり | △ Snowflake内部形式<br>移行時は書直し | × SF閉域<br>全面作直しリスク | ○ Delta Lake<br>+Databricks（松） |
| 原則3 | マルチLLM | ○ Bedrock経由<br>マルチLLM可能 | × Kozuchi/Takane<br>独自LLM | ○ Bedrock前提<br>マルチLLM可能 | △ Cortex経由<br>Claude/Llama等 | × Einstein独自<br>SF内で完結 | ○ GPT/Claude/Qwen等<br>実績多数 |
| 将来 | AI/ML開発環境・<br>将来拡張性 | ○ Agentic AI構成図<br>MLflow+Registry<br>Dev/Stg/Prod分離<br>Notebook統合環境 | △ DI Basic内蔵<br>詳細不明<br>環境分離の具体設計なし | × AI/MLは将来検討扱い<br>実験環境の提案なし | △ DataRobot MLOps<br>Snowflake内で完結<br>環境分離方法不明 | × 松プランのみ<br>Einstein Studio<br>独自閉域環境 | ○ Mosaic AI+MLflow<br>Agent Bricks<br>梅竹松で段階的環境提供<br>松でSandbox設計あり |

**補足：AI開発ロックインの評価基準**

× = 独自製品依存（乗り換え＝全面作り直し） … Kozuchi、Einstein等

△ = 特定PF依存（乗り換え可能だが書き直しコスト発生） … Snowflake内部形式等

○ = 業界標準 or OSS（移行先が複数あり現実的に乗り換え可能） … AWS標準、Delta/Iceberg等

# 6社RFI回答のレビュー（AI以外の観点）

各社のRFI回答をAI以外の観点で評価

| 評価項目 | Deloitte | Fujitsu | PwC | Hitachi | SF | Sky |
|---|---|---|---|---|---|---|
| コスト透明性 /コスト | ○（Sケア個別の試算詳細） | △(試算はないが、現時点では不透明なため妥当 ) | △ (概算のみ ) | △（Snowflake従量課金） | △ | ○（AWS明細型） |
| 既存資産の再利用可能性 | ○（PySpark native） | ○（PySpark native） | ○（Glue=PySpark可） | △（Snowpark、PySpark互換だが変換要） | × (PySpark非対応 ) | ○（Glue or Databricks） |
| ダッシュボード再現性<br>※Tableauで要件を満たすか要確認 | △（Tableau） | △（PowerBI/Tableau） | △（Tableau推奨） | △（Tableau/PowerBI対応） | △（Tableau） | △（QuickSight） |
| 運用負荷 | ○（マネージド中心、24/365グローバル保守体制） | △（Databricks主体だが、Sケア主体前提） | ×（AWS群の個別管 理） | △（SaaS構成だが複数ベンダー管理） | △ | ○（保守サービスの明記） |
| データ形式ロックイン | ○（Iceberg/Delta開放） | ○（Databricks主体） | △（AWS形式依存） | △（SF API依存） | ×（SF API依存） | △（フル AWS構成 ) |
| 移行リスク | ○（3パターン段階） | ○（egaku運用実績） | ○（3 Step移行） | △（介護業界 Snowflake実績あるが移行計画詳細不足 | △（情報不足） | ○ (実績多 ) |
| セキュリティ | ○（Unity Catalog+IAM） | ○（IAM+CloudTrail） | ○（IAM+Cognito） | ○（Horizon + HIPPA準拠 Snowflake実績） | ○（Einstein + Tableau RLS） | ○（IAM+CloudWatch, 松でUnity Catalog) |

# 6社RFI回答の評価まとめ

**RFI時点では、Deloitteが技術面で相対的に有力。**
**一方で比較継続候補にも固有の強み・確認事項があるため、最終判断は RFPで提案の具体化を求めたうえで行う**

| 会社 | 現時点の評価 | 総括 | 主な懸念/確認事項 |
|---|---|---|---|
| Deloitte | 有力候補 | Sケアデータに基づく詳細試算を提示。 OSSベースでロックイン回避。 AI/ML・非構造化・開発環境すべてに ○評価。24/365運用体制。技術面で穴がない唯一の提案。<br>ただし運用条件などは要確認 | 運用条件の具体性（マネージド体制の日本側要員数・対応範囲）<br>Dify等特定ツール依存 (databricks構成推奨なので大きな問題ではない ) |
| Fujitsu | 比較継続 | egaku/Foundry運用経験者が在籍する唯一の会社。現行環境を知っている価値は大きい。インフラ構成も具体的。「移せるか」の不確実性が最も低い。<br>ただしKozuchi/DIPのロックインとAI拡張性の弱さが懸念 | Kozuchi/DIPの独自ロックイン<br>インフラ構成は具体的だが、 Foundryパイプラインの移行マッピングは未提示 |
| Sky | 比較継続 | AI/ML構成提案が最も具体的（ Mosaic AI+MLflow+Agent Bricks）。マルチLLM実績も最多。コスト透明性が高い。将来の AI活用を見据えた設計力が強い。<br>ただし、Tableau非対応がリスク | Tableau非対応（QuickSight固定）<br>推進体制に具体性がない |
| Hitachi | 比較継続 | 介護業界の Snowflake実績あり。データ分析民主化の推進計画が具体的。業界理解とユーザー育成の視点は他社にない。<br>ただしSnowflake単一、PySpark移行の具体計画なし、オープン標準への言及ゼロ | Pyspark移行の工数・実現性<br>オープン標準への言及ゼロ<br>統合カタログの言及なし |
| PwC | 比較継続 | SOMPOグループとの関係性、ガバナンス設計の具体性、コンサル力は強い。「仕組みづくり」の伴走者としては最適。<br>ただしAWS単体運用で運用負荷高 | 運用負荷が高い (AWS郡の個別管理 )<br>OSSを明示的に拒否。 |
| SF | 優先度低 | データ基盤としてはオーバースペック（年 62M～248M円）。AI機能は最高額プラン限定。SF閉域のロックイン。 CRM連携が主目的なら検討余地があるが、全社データ基盤としては不適合 | 全社データ基盤としてはオーバースペック（年 62M～248M円）。AI機能は松プラン限定。 SF閉域のロックイン |

# ベンダー選定時の RFP評価観点・依頼事項

RFI評価から特に注視すべき評価観点と依頼事項を整理する

| RFPで特に確認すべき観点 | RFP依頼事項 |
|---|---|
| **移行実現性** | Foundryパイプラインの移行/ 再構築の進め方、再利用可能範囲、書き換えが必要な範囲、想定工数、類似実績があれば提示すること。あわせてBIツールの選定方針と現行Tableau/Foundry UI資産の再利用/移行方針を記載すること |
| **コスト妥当性** | 提示条件(※参照)に基づき、初期費用・月額/ 年額費用・従量課金の内訳と算出根拠を提示すること。未提示の前提条件を置く場合は、その想定値と根拠を記載すること。 |
| **運用負荷** | マネージド/アンマネージド運用、必要に応じてハイブリッド運用の対応可否を示すこと。それぞれについて、役割分担、メリット・デメリット、コスト差分を記載すること。 |
| **オープン性** | 採用するデータ形式(Iceberg/Delta等)を記載すること。標準形式でない部分、制約事項、再構築や追加対応が必要となる箇所があれば明示すること。 |
| **カタログ** | カタログ管理対象（例：データセット、パイプライン、ダッシュボード、非構造データ）と、リネージの追跡範囲（例：取込元から加工・配信先まで）を記載すること。あわせて、カタログの自動取得 手動設定 / 自動更新の対象範囲・運用方針・制約事項を明示すること。 |
| **AI拡張性**<br>**※参考として Sケア・HDのAI検討状況も提示** | AI開発環境環境の構成案、OSS / 業界標準 / 独自製品のどこに依存する構成かと採用理由・実績を記載。環境分離方針、利用可能データ範囲、PoCから本番移行までの前提条件・制約事項を記載すること。採用する各ツールについて、代替可能な選択肢があれば併記すること |
| **LLM柔軟性** | 利用可能なLLM(gpt/claude等)と制約事項があれば記載すること。 |
| **プロジェクト体制** | 本PJ向けの体制図(役割・人数・主要メンバーの経歴・経験)を記載すること。 |

**※コスト試算前提**
Pipeline：219本 / Row Data合計：約2.35TB / Summary Data：約101GB / ダッシュボード：15画面 / BI閲覧者数：約3,000名 / 外部連携：Tableau7、施設300 / 月間アクセス数：約2万アクセス

# AI活用に向けた将来論点

AI活用の進展を見据え、将来追加で検討が必要となる論点を7つの観点で整理する

| 観点 | 将来論点 | 将来の検討事項 | いつ検討すべきか |
|---|---|---|---|
| ①連携性 | HD/グループ会社とのデータ共有や、施設 IoT・映像等のリアルタイム連携が必要になる可能性がある | 共有方式、リアルタイム連携要否、対象データ、更新頻度、遅延許容、障害時再送 | HD/グループ会社側の連携方針が確定した時、またはIoT/映像ユースケースが具体化した時 |
| ②安全性 | AIがデータアクセス主体となる場合、人と同等の ID・権限・監査・ガードレール設計が必要になる | AI主体を含む認可方針、データ分類、監査方針、AI個別権限、入出力制御 | AIユースケース検討時 |
| ③進化性 | AIモデルは短期間で変化するため、精度評価・劣化検知・改善サイクルを前提とした運用が必要になる | 評価指標、劣化検知、モデル /プロンプト/RAG構成の管理、切替手順、改善サイクル | 本番利用するAI機能・モデルの導入方針が決まった時 |
| ④実効性 | 介護記録テキスト、音声、 IoT、映像等の非構造化データは、構造化データと異なる加工設計が必要になる | 対象データ選定、原本 /加工後データ管理、チャンク化・ベクトル化、品質管理、構造化データとの紐付け | 非構造化データを用いる具体ユースケースが決まり、対象データと精度要件が明確になった時 |
| ⑤運用性 | AIはモデル・データ・プロンプト・ツールの複合体となるため、障害時の原因切り分け、停止判断、コスト管理が難しくなる | ログ/トレース、障害切り分け、停止基準、コスト可視化、監視・運用体制 | AIユースケースの実装設計に着手する時 |
| ⑥開発速度 | AI開発は試行錯誤前提であり、 PoC→本番→横展開の標準手順が未整備だと立ち上げが遅延する | 標準運用、利用可能データ範囲、権限付与、PoC→本番移行条件 | PoC候補具体化時に環境要件を確定 |
| ⑦規模 | AI/Agent増加で品質・責任・コスト管理が難しくなる | 登録管理、標準化、責任分界、コスト管理 | 複数AI機能が開発された時 |

※ 体制・人材・制度の論点も別途存在

# SW構成基本方針/データ・フロー基本方針

# 主要コンポーネントの基本構成方針

AI活用の進展も見据えつつ、機能だけではなく運用実現性も考慮した方針を採用する

| コンポーネント | 構成方針 |
|---|---|
| **インテグレーション** | •多様なデータソースを標準的な接続方式で取り込める構成とする<br>•既存連携の再利用を重視しつつ、将来の非構造化データや外部サービス連携の追加にも対応可能とする |
| **ストレージ** | •構造化データを中心に、原本・標準化済み・活用用データを統合管理できる構成とする<br>•将来の非構造化データ追加を阻害しないことを前提とする |
| **オーケストレーション** | •ワークフロー実行、スケジュール管理、再実行、監視を一元的に扱える構成とする<br>•個別運用を増やしすぎず運用負荷を抑えやすい構成とする |
| **変換・処理** | •既存パイプライン資産の再利用性・移植性を重視し、処理ロジックを継承しやすい構成とする<br>•将来の追加加工 (非構造データ含む )にも拡張可能とし、管理しやすい構成とする。個別実装や属人運用を増やさないことを重視する |
| **ガバナンス** | •カタログ、リネージ、権限、監査ログを統合的に管理・運用できる構成とする<br>•データとパイプラインを横断して可視化・統制でき、管理対象、更新責任、監査運用が継続できることを重視する |
| **ML/AI** | •ML/AI機能そのものは現時点で存在しないため、実装可能な枠確保を基本とする<br>•将来のML・AIモデル、プロンプト /RAG、推論ログを追加しやすい構成とする<br>•将来的に特定モデルのみや特定技術に限定されない構成とする |
| **可視化** | •既存BI資産の継続活用は必須としないが、移行後に可視化のクオリティや活用面で負担がない表示が可能な構成とする |

# データの段階的な階層設計

Bronze（生データ）→ Silver（クレンジング済）→ Gold（ビジネスロジック適用）の 3層構造

| 設計基準 | Bronze層 | Silver層 | Gold層 |
|---|---|---|---|
| 目的 | **ソースデータの保管<br>「事実の記録」として不変** | **分析可能な状態に標準化<br>クレンジング・正規化済** | **ビジネス KPI・BI/ダッシュボード接続用<br>集計・結合・ロジック適用済** |
| 変換ルール | 取り込んだままを保持 | 型変換・NULL処理・重複排除<br>カラム名の標準化 | 集計・結合・ビジネスロジック |
| 品質チェック | 到着確認・件数チェック・スキーマ確認 | 品質ゲート（違反行はアラート） | KPI整合性チェック |
| テーブル命名例 | raw_care_records<br>raw_sensor_data | clean_care_records<br>master_facilities | kpi_facility_monthly<br>dim_staff, fact_care_events |

**設計原則：各層の独立性を維持し、 Bronze → Silver → Goldの一方向依存を徹底**

階層に分離することで不具合時の原因特定を明確化。 BIツール利用ユーザーには Gold層を提供、分析ユーザーには Silver層を提供。

# 開発・本番環境の分離

安定運用のための環境分離の方針

## 3階層の環境構成

| 開発環境 (dev) | ステージング環境 (stg) | 本番環境 (prod) |
|---|---|---|
| **開発者個人の実験・検証** | **統合テスト・品質検証** | **本番環境・ダッシュボード接続** |
| Branch単位で作業 | 本番同等スキーマで検証 | 変更はStagingからのリリースのみ |
| サンプルデータを使用 | CI/CDで自動デプロイ | 本番データ・施設別アクセス制御 |

**原則：環境間のデータ移動は禁止。コード（パイプライン定義）のみが　CI/CDで環境間を移動する**

Dev環境では特定時点のスナップショットのサンプルデータを使用して、開発に活用。　Stagingで本番同等の環境で検証を行い、合格した場合に　Prodへリリース。

## 開発～リリースの流れ

**① 開発・実験**
開発環境

- Feature Branchで作業
- サンプルデータで開発・検証
- テスト実行

**② PR・レビュー**
コードレビュー

- PR作成（最低1名承認）
- テスト自動チェック
- mainブランチへMerge

**③ ステージング反映・検証**
ステージング環境

- CI/CDで自動デプロイ
- 本番同等データで統合テスト
- データ品質チェック通過を確認

**④ 本番リリース PR**
本番リリースのレビュー

- リリース承認（担当者）
- CI/CDで自動デプロイ

# Appendix 目次


## 別紙A アーキテクチャ・技術選定

A-1 アーキテクチャ: DWH+DL vs Lakehouse
A-2 テーブルフォーマットとは
A-3 UniFormによるフォーマットの収束
A-4 プラットフォーム比較: Databricks vs Snowflake vs AWS
A-5 本プロジェクトにおける各 PF評価
A-6 BI層にSnowflakeを挟む必要性
A-7 オーケストレーション: 構成パターン比較
A-8 データ変換: Foundry PySpark移植戦略
A-9 セマンティックレイヤー: Ontology代替

## 別紙B ガバナンス移行マッピング

B-1 データガバナンス: Foundry機能の代替マッピング

## 別紙C 推奨技術スタック総括

C-1 推奨技術スタック: 総括
C-2 技術選定サマリー

# 1 アーキテクチャ:「 DWH + Data Lake」vs「Lakehouse」

従来の2層構成を1つに統合した Lakehouse が新しい標準

## 従来型: DWH + Data Lake（2層構成）

**Data Lake**
S3 / GCS
生データを安価に保管
SQLで直接分析は不可

→ ETL →

**Data Warehouse**
Snowflake / Redshift
加工済みデータをBI向けに提供
コスト高・ML/AI非対応

- ✗ データが2箇所に存在 → 同期コスト・整合性の問題
- ✗ DWHライセンス + Lake運用 = 二重課金
- ✗ ML/AIはDWHから取り出して別環境で実行が必要

## Lakehouse（統合型 — 推奨）

**Lakehouse**
Databricks + S3 / Palantir Foundry（現行）

| 保管 | 分析 | 加工 | AI | 管理 |
|---|---|---|---|---|
| Data Lakeの安価なストレージ | DWHの高速なSQL検索 | ETLパイプラインを統合実行 | ML/AIを同一基盤で | 権限・監査リネージ統合 |

- ✓ データは1箇所 → 管理がシンプル・整合性の問題なし
- ✓ ストレージはS3価格 + コンピュートは従量課金
- ✓ BI・ML/AI・ETLが同一基盤で完結

| 観点 | DWH + Data Lake（2層） | Lakehouse（統合） |
|---|---|---|
| データの場所 | 2箇所（Lakeに保管 → DWHにコピー） | 1箇所（S3上にすべて統合） |
| コスト構造 | DWHライセンス + Lake運用 = 二重課金 | ストレージはS3価格 + コンピュート従量課金 |
| BI対応 | ◎ DWH側は非常に高速 | ○ 近年大幅改善。実用上十分 |
| ML/AI対応 | △ DWHから取り出して別環境で実行 | ◎ 同一基盤でデータに直接アクセス |
| ガバナンス | ○ DWHとLakeで別々に管理 | ◎ 統合管理（権限・監査・リネージ） |
| ベンダーロックイン | △ DWH側にロックインされやすい | ◎ オープンフォーマット（iceberg等）で回避可能 |
| 市場トレンド | レガシー構成 | 新規構築の主流 |

# 2 テーブルフォーマットとは

S3上のファイルを「テーブル」として扱うための仕組み

## テーブルフォーマットなし（S3のみ）

**S3バケット**

- data/year=2024/month=01/part-001.parquet
- data/year=2024/month=02/part-002.parquet
- data/year=2024/month=03/part-003.parquet
- ...

→ 単なるファイル置き場

→ SQLでの検索・更新・削除が不可能

## テーブルフォーマットあり（Iceberg/Delta）

**S3 + メタデータ層**

✓ スキーマ管理 — 列の追加・変更を安全に実行

✓ ACIDトランザクション — 途中で壊れない書き込み保証

✓ Time Travel — 過去のデータに遡れる（監査対応）

✓ パーティション管理 — 高速検索のための自動分割

✓ 行レベルの更新・削除 — RDBのようなUPDATE/DELETE

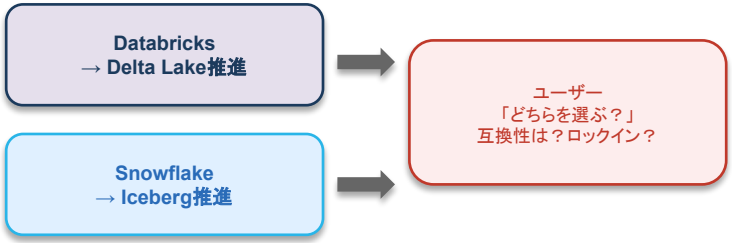
## 主要フォーマット 3種の位置づけ

| | Apache Iceberg | Delta Lake | Apache Hudi |
|---|---|---|---|
| **起源** | Netflix → Apache財団 | Databricks → Linux Foundation | Uber → Apache財団 |
| **位置づけ** | 業界標準の互換フォーマット | Spark環境での最高性能 | ストリーミング特化（ニッチ） |
| **採用状況** | 975社以上（2026年）<br>Snowflake/Athena/BigQuery対応 | Databricksエコシステム中心<br>Spark上での性能は最速 | 採用企業は限定的<br>リアルタイムUpsertに強い |
| **相互運用** | マルチエンジン対応が最も広い<br>ベンダー中立性◎ | UniFormでIceberg互換を実現<br>Databricks利用時のデフォルト | Iceberg互換を自ら追加<br>本PJでは選択肢外 |

# 2 補足：UniFormによる「フォーマット」の収束

Delta Lakeで書き込み → 自動でIcebergメタデータ生成 → 外部エンジンからIcebergとして読取り可能

## 以前の状況：二者択一




## 現在：UniForm（2024年6月GA）で解決

## UniFormの制約事項（要認識）

| 制約 | 内容 | 影響度 |
|---|---|---|
| **読み取り専用** | 外部エンジンからの書き込み不可。書き込みはDatabricks/Delta経由のみ | 中 |
| **メタデータの遅延** | Icebergメタデータは非同期生成。書き込み直後は一瞬のタイムラグあり | 低 |
| **性能ペナルティ** | Snowflakeからの読み取りはネイティブ比で約20%の性能低下 | 中 |
| **Databricks固有機能** | UniFormはオープンな業界標準ではなくDatabricks固有の機能 | 要認識 |

**結論：Databricks利用時：Delta Lake + UniForm（Iceberg互換）を推奨。**

Databricksを書き込みプラットフォームとする前提で二者択一が不要に。マルチプラットフォーム時：Icebergを標準化。ベンダー中立性が最も高い。

# 3 プラットフォーム比較：Databricks vs Snowflake vs AWS Native

別紙A-4

PySpark互換性・AI拡張性・ガバナンスの観点で評価

**Databricks**
データエンジニアリング + AI の統合環境

**Snowflake**
SQLの王者。シンプルで速い

**AWS Native**
部品を自分で組み立てる最安構成

| 評価軸 | Databricks | Snowflake | AWS Native |
|---|---|---|---|
| **PySpark互換性** | ◎ そのまま動く | × Snowparkへ書き換え | ○ EMR/Glueで動く |
| **AI/ML** | ◎ MLflow, Vector Search等 統合 | △ Cortex AI 限定的 | ○ SageMaker+Bedrock |
| **SQL分析性能** | ○ 十分高速 | ◎ 最速クラス | ○ Athena/Redshift |
| **ガバナンス** | ◎ Unity Catalog統合管理 | ◎ Horizon（SF内は強い） | △ Lake Formation |
| **オープン性** | ◎ UC(コアOSS), Delta+Iceberg | △ 独自形式+Apache Polaris | ○ OSSベース |
| **運用負荷** | ○ サーバーレス化進行中 | ◎ フルマネージド | × 自前構築・運用 |
| **コスト構造** | 中～高（DBU+インフラ二重） | 中（クレジット制） | 低～中（従量+人件費） |
| **必要人材** | 2名～（データエンジニア） | 1-2名（SQL人材） | 3-4名（インフラ+データ） |

# 3 本プロジェクトにおける各プラットフォームの評価

Foundry移行の要件に照らした適合度

**Databricks**
◎ 最も適合度が高い

**Snowflake**
△ 本プロジェクトでは不利

**AWS Native**
○ 安いが運用が重い

| 評価軸 | Databricks | Snowflake | AWS Native |
|---|---|---|---|
| PySpark互換性 | ◎ そのまま動作<br>（デコレーター除去のみ） | × Snowparkへ全面書き換え<br>RDD/UDF/ML全て移植が必要 | ○ EMR上で動作可能<br>ただしGlue PySpark は制約あり |
| **AI/ML拡張性** | ◎ MLflow, Vector Search<br>Model Serving, Feature Store | △ Cortex ML は発展途上<br>Databricksに比べ機能限定的 | ○ SageMaker連携可能<br>ただし統合度は低い |
| **ベンダーロックイン** | ○ Unity Catalog OSS化<br>で懸念が大幅に軽減 | × 独自形式でデータ保持<br>エクスポートにコスト発生 | ◎ OSSベース<br>S3 + Iceberg で中立性高い |
| **運用負荷** | ◎ 統合プラットフォーム<br>Foundryに最も近い体験 | ◎ フルマネージド<br>運用負荷は最小 | × 6-8個のサービス統合<br>障害切り分けが複雑 |
| **人材確保** | ○ Spark/Python人材<br>市場に比較的多い | ○ SQL中心で敷居は低い<br>ただしSnowpark人材は少数 | △ AWSインフラスキル<br>全サービス横断の人材が困難 |
| **コスト** | ○ 中程度<br>Commit割引で最適化可能 | △ 高め<br>PySpark書き換え工数も加算 | ○ 最安<br>ただし人件費含むと差は縮小 |

**結論：本プロジェクトでは Databricksを推奨**

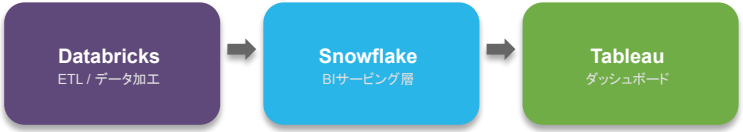
PySpark互換性、AI拡張性、Unity Catalogのオープン性が、Foundry移行の要件に最も合致する。
SnowflakeはPySpark書き換えコスト大、AWS NativeはOSS運用工数が見えにくいリスクあり。

# 3 BI層にSnowflakeを挟む必要性

よくあるパターン（Databricks ETL + Snowflake）は必要か？

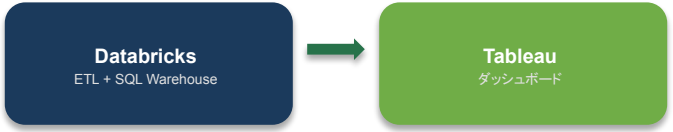
**以前はTableau同時多数接続に Snowflakeを挟むメリットがあったが、現在は性能差は実運用では少なく、コストと手間だけ重くなる設計。**

### パターンA：Snowflakeを挟む構成




⚠️ Snowflake追加コスト：年間 1,500-3,000万円（ライセンス ＋ 同期パイプライン構築・運用）

### パターンB：直接接続（推奨）




✓ シンプル。追加コストなし。年間 1,500-3,000万円を削減

## ポイント：「登録ユーザー数」と「同時接続数」は全く異なる

| 指標 | 推定値 | 根拠 |
|---|---|---|
| 登録ユーザー | 800 - 1,000人 | 施設長700名 + 本部スタッフ |
| 日次アクティブ率 | 20 - 30% | 施設長は毎日閲覧しない。週次月次チェックが多い |
| ピーク同時接続 | 40 - 80人 | 朝の業務開始時、月初レポート確認時が最大 |
| 平常時同時接続 | 10 - 30人 | — |

### Databricks SQL Serverlessの同時実行能力

10並行クエリ/クラスター × 自動スケーリング
ピーク80人 → 8クラスターで対応（自動）
上限: 1,000並行クエリ — 全く問題なし
**2025年: 実ワークロード 5倍高速化を達成**

### さらにTableau Extractで同時接続問題自体が消える

4ダッシュボードはいずれも日次バッチ更新で十分
Extract: 定期的にデータをTableau Serverにキャッシュ
800人がアクセスしてもDatabricksへの負荷はゼロ
**Snowflakeを挟む理由が完全になくなる**

# 4 オーケストレーション：構成パターンの比較

Databricks中心であればLakeflow単体が最もシンプルかつ推奨。外部連携の複雑度に応じて構成を選択

パターン1

## Lakeflow 単体 (databricks)

推奨

**Lakeflow Jobs**
- バッチETL
- ストリーミング
- ML推論

**Lakeflow Pipelines**
- 宣言的ETL
- 品質チェック

パターン2

## Lakeflow + Airflow

外部連携が複雑な場合

**Lakeflow Jobs**
- Databricks内ETL

**Airflow DAG**
- 外部API連携
- クロスシステム依存

パターン3

## Airflow/Dagster 一本化

統一管理を重視

**外部オーケストレーター**
- 全ジョブ統合管理

**Databricks API呼出**
- Spark Job投入のみ

| 評価軸 | パターン1：Lakeflow単体 | パターン2：Lakeflow + Airflow | パターン3：外部一本化 |
|---|---|---|---|
| 運用負荷 | ◎ 最小（フルマネージド） | △ 2ツール運用が必要 | △ 外部ツール運用が増加 |
| Unity Catalog統合 | ◎ ネイティブ統合 | ○ Databricks内は統合 | △ API経由のみ |
| 外部システム連携 | △ 複雑な場合に不便 | ◎ Airflowで柔軟に対応 | ◎ 全システム一元管理 |
| セットアップ | ◎ 追加構築不要 | △ Airflow環境の構築が必要 | △ 専用環境の構築が必要 |
| コスト | ◎ Databricks内で完結 | ○ Airflow運用コスト追加 | ○ 外部ツールコスト追加 |

**結論：Databricks中心であればLakeflow単体を推奨**

フルマネージドで運用負荷が低く、Unity Catalogとの統合も深い。外部システム連携が複雑な場合のみAirflowを追加。

# 5 データ変換: Foundry PySpark パイプラインの移植戦略



既存のPySparkコードはDatabricksへほぼそのまま移植可能。追加ツールの導入は不要

## 移植の全体像

**現行: Foundry**

@transform_df
@configure
Foundry固有デコレーター
PySpark DataFrame API

→ デコレーター除去のみ

**移行先: Databricks**

PySpark DataFrame API
（そのまま動作）
Lakeflow Pipelines
Databricks Notebooks

## Databricksが標準提供する機能（追加ツール不要）

| | |
|---|---|
| IDE / 開発環境 | **Databricks Notebooks + SQL Editor**<br>Foundry Code Repositoriesの代替 |
| ジョブ管理 | **Lakeflow Jobs（サーバーレス）**<br>Foundryビルドシステムの代替 |
| 宣言的ETL | **Lakeflow Declarative Pipelines**<br>パイプライン定義・自動実行 |
| CI/CD | **Databricks Asset Bundles + GitHub Actions**<br>Code RepositoriesのPR→ビルドの代替 |
| リネージ | **Unity Catalog（PySpark列レベル対応）**<br>Foundry自動リネージの代替 |
| テスト | **Databricks DLT Expectations**<br>データ品質チェックの自動化 |

## Foundry → Databricks: 機能対応マッピング

| Foundry機能 | Databricksでの対応 | 移植工数 | 備考 |
|---|---|---|---|
| **PySpark パイプライン（@transform_df等）** | PySpark on Databricks | 低<br>デコレーター除去のみ | コアロジックはそのまま動作 |
| **Code Repositories（Git + ビルド一体化）** | GitHub + Databricks Asset Bundles<br>+ Lakeflow Jobs | 中<br>CI/CDパイプライン構築 | RFP提案にこの工数が含まれているか要確認 |
| **ブランチベースのデータ開発** | スキーマ分離 + 開発/本番<br>プロモーションフロー | 中～高<br>運用フロー再設計 | Foundry最大の差別化機能。完全代替は困難<br>開発→本番の運用ルール再設計が必要 |
| **自動データバージョニング** | Iceberg/Delta Time Travel<br>+ VACUUMポリシー設計 | 低<br>保持期間の設計のみ | 「全ビルド無期限保持」は非現実的<br>スナップショット保持ポリシーの設計が必要 |
| **自動データリネージ** | Unity Catalog Lineage<br>（PySpark列レベル対応） | なし<br>自動で動作 | .toPandas()やRDD操作のみ非対応 |

# 6 セマンティックレイヤー: Ontology代替の要否

ビジネス用語の定義・計算ロジックを一箇所で管理する層。FoundryのOntologyに相当

### セマンティックレイヤーなし

- **Tableau** — 入居率 = 入居者数 / 定員（定義A）
- **AIチャット** — 入居率 = 入居者数 / ベッド数（定義B（微妙に違う））
- **Excelレポート** — 入居率 = ???（定義C（不明））

→ 数値が合わない。信頼を失う

### セマンティックレイヤーあり

**セマンティックレイヤー**
入居率 = 入居者数 / 定員（唯一の定義）

- Tableau
- AIチャット
- Excel

→ すべて同じ数値。Single Source of Truth

## 必要性の判断基準

| ユースケース | セマンティックレイヤー | 理由 |
|---|---|---|
| **ダッシュボード4つのみ（現状維持）** | 不要（後から追加可能） | Tableauの計算フィールドで十分 |
| **ダッシュボード増加（10+）** | 推奨 | 計算定義の食い違い防止 |
| **AIチャット（自然言語分析）** | 実質必須 | セマンティック層によりAIのSQL精度が約3倍向上<br>（学術研究 16%→54%） |
| **予測AI + 自動レポート** | 必須 | Feature Storeとの連携も必要 |

**主な選択肢:**

- **Databricks Metric Views** UC統合・GA済み
- **dbt MetricFlow** OSS (Apache 2.0)
- **Cube.dev** OLAP高速化

**結論：初期構築では不要。AIチャットをスコープに入れる場合は初期から設計が必要。**

Databricks利用時はMetric Views（Unity Catalog内蔵、2025年後半GA）が最もシンプルな選択肢。

# 7 データガバナンス: Foundry機能の代替マッピング

Foundryで統合的に動作していたガバナンス機能を、Unity Catalogでどう代替するか

| Foundry機能 | 役割 | Unity Catalogでの代替 | 対応度 | ギャップ / 注意点 |
|---|---|---|---|---|
| Markings（タグ自動伝播） | データ分類タグの自動伝播 | Tags（手動 or API設定） | △ | 自動伝播なし— CI/CDでタグ付け<br>ルールを自動化する運用設計が必要 |
| RLS（行レベルセキュリティ） | 施設ごとのアクセス制御（700施設） | 行フィルタ + 列マスキング | ◎ | 対応可能。動的RLSも実装可<br>Foundryと同等のきめ細かい制御 |
| 自動リネージ | データの来歴を自動追跡 | SQL: 自動追跡<br>PySpark: 自動追跡（GA済） | ○ | PySpark lineageはUnity Catalogで<br>自動追跡。toPandas/RDDは非対応 |
| 監査ログ | 誰が何にアクセスしたかの記録 | System Tables<br>（Audit Logs） | ◎ | 対応可能。SQLで柔軟に分析可<br>Foundryより分析の自由度が高い |
| PII検出（Sensitive Data Scanner） | 個人情報の自動検出・マスキング | Presidio（OSS）<br>or AWS Macie | △ | 別途導入・設定が必要<br>介護データは要配慮個人情報— 必須 |

**Unity Catalogの位置づけ**

コアカタログ機能はApache 2.0でOSS化。ただし高度なガバナンス（列リネージ、動的ポリシー、PII分類）はDatabricks商用版が必要。

**◎ そのまま代替可能**

- RLS
- 監査ログ

**○ 運用設計で対応**

- Markings
- リネージ

**△ 別途ツール導入**

- PII検出

# 8 推奨技術スタック：総括

Foundry移行における推奨構成

| | |
|---|---|
| BI / 分析 | Tableau（既存維持） |
| セマンティック | 初期は不要 → AI導入時 Metric Views等を検討 |
| データ変換 | PySpark（Python）+ dbt Core（SQL） |
| オーケストレーション | Lakeflow Jobs（+ 必要時 Airflow） |
| ガバナンス | Unity Catalog（データ + ML + AI 統合管理） |
| テーブルフォーマット | Delta Lake + UniForm（Iceberg互換） |
| ストレージ | S3 + Lakehouseアーキテクチャ |

**基盤：Databricks Lakehouse Platform**

## この構成のメリット

1. **PySpark移植コスト最小**
   Foundryデコレーター除去のみでほぼ動作
2. **ツール数最小**
   Databricks + dbt + Tableau の3つが中核
3. **ロックイン回避**
   UC(OSS) + Delta+UniForm(Iceberg互換)
4. **AI拡張性**
   MLflow, Vector Search, Model Serving統合済み
5. **運用人材**
   2名程度（データエンジニア中心）で運用可能

### リスク・注意点

- DBU単価の不透明性 — 見積段階で開示を要求
- Markings自動伝播の欠如 — CI/CD自動化で補完
- ストレージ膨張 — VACUUMポリシーの設計必須